

LLM Agents!

Jon Trnka

2026-03-09

Table of contents

1	Introduction	2
2	Objective:	2
2.1	What exactly is an LLM Agent?	2
2.2	LLM-Agent vs. Non-LLM-Agent AI comparisons + examples	3
3	Specific Aims	4
4	Data	4
4.1	My data will be from 1 non-api and 3 different API sources.	4
4.2	Main research question:	4
5	Methods and Research Design	5
5.1	Experimental Framework	5
5.2	Task Selection	5
5.3	Research Planning:	5
5.4	Tool and Document design:	6
5.5	How it all comes together:	6
5.6	Before Diving in:	6
5.7	'Does the complexity(# of tool calls required to complete a task) of the task predict total_latency?	8
5.8	'Does the number of tools(actual # of tool calls the agent executed) used predict total task latency?	10
5.9	Does API difficulty(Calculator:1, Web search: 2, Weather: 3) predict total_latency?	11
6	Concluision:	12
7	For those wanting more!	12
8	Disclaimer:	13
9	Questions, comments, suggestions?	13
	References:	13

1 Introduction

Hello! My name is Jonathan Trnka and this is my presentation on LLM Agents!

2 Objective:

- Quickly explain what an LLM is
- Explain what makes an LLM-Agent different
- Compare them
- Explain my process
- Use statistical tools to explain my Data
- Conclusion

2.1 What exactly is an LLM Agent?

- To put it simply, an LLM Agent is the next step in evolution of LLMs.
- What is an LLM aka Large Language Model?
 - As Zhu et al. (2025) states, “LLM chatbots are designed as reactive conversational systems that receive textual prompts from users, process these inputs using pretrained knowledge, and generate coherent textual responses. Typical use cases include open domain dialogue, customer support, question answering (Zhu et al.). Despite LLMs being very powerful in their own way, they are very limited by what they can do. Some common issues is outdated information and hallucinated responses.
- What’s a hallucinated response you say?
 - A hallucinated response in regards to LLMs is when it creates incorrect, or fabricates content that believes is correct when it is not. For example, you could ask for a 5 step plan and it gives you 4 believe it to be correct. It’s all based on the prompt given to it by the user.
- What does the ‘agent’ part mean?
 - The ‘agent’ part corresponds with the autonomous part of an LLM-Agent. This allows the LLM-Agent to resolve tasks on its own, without further input from a user. It can access APIs to access certain content, anything on the web, and solve errors it runs into.
- What is RAG?
 - RAG which stands for Retrieval Augmented Generation, is the solution to the limitations of standard LLMs. This is done by drawing real time data from the sources such as APIs. Ferrag et al. (2025)
 - Limitations such as outdated information, single prompt answers, no recovery plans and so on.

2.2 LLM-Agent vs. Non-LLM-Agent AI comparisons + examples

2.2.1 Table side-by-side Comparison

Aspect	LLM-Agent	Non-LLM-Agent
Task Handling	Multi-step planning	Single-shot answer
Tool Use	Dynamic, context-driven	None
Accuracy	High on factual & numeric tasks	Variable; prone to errors
Adaptability	Adjusts plan mid-run	Fixed output
Error Recovery	Retries or switches tools	No recovery
Transparency	Shows steps & traces	Opaque
Latency	Slower	Fast
Cost	Higher	Low
Best For	Complex tasks	Simple tasks

2.2.2 Types of LLMs and LLM-Agents.

Category	Model / System	Description	Agentic?
Standard LLM	GPT-4o / GPT-4 Turbo	General-purpose chat models; strong reasoning but no built-in planning.	No
Standard LLM	Claude 3 Opus / Sonnet	High-quality reasoning and writing; non-agentic unless wrapped in a system.	No
Standard LLM	LLaMA-3.1 / LLaMA-3.2 / LLaMA-3.3	Open models used in research; non-agentic by default.	No
Standard LLM	DeepSeek-V3	Efficient, high-performance model; not agentic without a tool layer.	No
Standard LLM	Mixtral 8x7B / 8x22B	Sparse MoE models; strong performance but no autonomy.	No
LLM-Agent System	OpenAI o-series with function calling	Uses planning + tool calls + multi-step execution when orchestrated.	Yes
LLM-Agent System	Google Gemini 2.0 + Tools	Can plan, call APIs, and execute multi-step tasks when tools are enabled.	Yes
LLM-Agent System	Groq LLaMA-3.3 + my agent pipeline	Becomes agentic through my code of planning loops, tool calls, and error recovery.	Yes
LLM-Agent System	LangChain Agents	Framework that wraps any LLM with planning, tools, and autonomy.	Yes
LLM-Agent System	Microsoft Autogen Agents	Multi-agent orchestration layer enabling planning and tool use.	Yes
LLM-Agent System	ReAct-style LLM Agents	LLMs using reasoning + action loops to solve tasks step-by-step.	Yes

3 Specific Aims

My goal is to evaluate how effective a LLM-Agent system is compared to a baseline LLM. To do this, I will construct two models: an LLM-Agent capable of multi-step planning, tool use, API integration, and limited error recovery, and a baseline LLM that processes only one prompt at a time without tool use or autonomous decision-making. I will benchmark both systems on the same set of tasks and use linear regression techniques to analyze differences in performance, error rates, and execution time.

4 Data

4.1 My data will be from 1 non-api and 3 different API sources.

- Tavily (web search base)
- OpenWeather
- Calculator
- Groq (the brain)

4.2 Main research question:

“How well can an LLM-agent perform multi-step tasks, self-recovery, accessing API keys, replanning actions, compared to a baseline LLM that does not?”

4.2.1 Sub-research questions:

- Does the complexity (# of tool calls required to complete a task) of the task predict total_latency?
 - $Y = \text{total_latency}$
 - $X = \text{task complexity (\# of tool calls required to complete a task)}$
- Does the number of tools (actual # of tool calls the agent executed) used predict total task latency?
 - $Y = \text{total_latency}$
 - $X = \text{task type}$
- Does API difficulty predict total_latency?
 - $Y = \text{total_latency}$
 - $X = \text{API difficulty (numerical: calculator: 1, web-search: 2, weather: 3)}$

5 Methods and Research Design

5.1 Experimental Framework

5.1.1 The study uses a between-systems comparative design. Two AI systems are constructed:

- LLM-Agent — a system capable of multi-step planning, tool use, API calls, and limited error recovery. It can autonomously break down tasks, select tools, and retry operations when failures occur.
- Baseline AI — a non-agentic model restricted to single-prompt, single-response interactions. It cannot use tools, plan, or recover from errors. It behaves like a standard chat assistant.
- Both systems are evaluated on identical tasks to isolate the effect of agentic capabilities.

5.2 Task Selection

5.2.1 Tasks are chosen to require reasoning, multi-step execution, or external information retrieval. Each task is designed so that:

- The LLM-Agent system can leverage planning and tools.
- The baseline system must attempt the same task without those capabilities.
- Success, failure, and time-to-completion can be measured consistently.

5.2.2 Examples include:

- Multi-step problem solving (e.g., weather + calculation + summarization).
- API-dependent tasks (e.g., retrieving structured data).
- Error-prone tasks where retries matter (e.g., handling missing parameters).

5.3 Research Planning:

- Identify what my main research question would be and create my sub-research questions. All of which evaluate the performance of an LLM-Agent vs non-LLM-Agent (baseline)
- Select and test my chosen variables.
 - task_complexity
 - api_difficulty
 - total_latency
 - tool_count
 - error count

5.4 Tool and Document design:

- Chose tools that would be show multi-step agentic behavior. This would be weather retrieval, calculator functions, and web search.
- Gather the required API keys and create a .ENV file. This would allow seamless access when executing code without hard-coding the credentials.
- Create my 'Agents_Multi_step' python file. This holds all core functionality such as calling API keys, tool definitions, LLM-Agent and baseline LLM decision making, log functions, and more.
- Create my 'LLM_Agent_pipeline' python file. This one holds code for running statistical analysis, creating summary tables, the plots, the legends and more.

5.5 How it all comes together:

1. Benchmark Queries (36 total)
2. Run Agent + Baseline
 - Agent: multi-step, may call 0-many tools
 - Baseline: single LLM call
3. Write Raw Logs
 - agent_runs.jsonl (one row per tool call)
 - baseline_runs.jsonl (one row per query)
4. Load JSONL Logs inside the .qmd pipeline
5. Clean + Aggregate
 - collapse retries
 - compute total latency
 - extract final answers
 - normalize errors
 - count tool usage
 - produce 1 row per query
6. Export Clean Results to CSV summaries:
 - agentic_results.csv
 - baseline_results.csv
7. Quarto Loads CSVs for regressions, plots, comparisons, and legends
8. Generate Final PDF (plots, regressions, comparisons, summaries)

5.6 Before Diving in:

- 6 questions for each type of query listed:
 - Math
 - weather
 - Web search

- Multi Tool
 - Failure-recovery
 - Ambiguous/complex
- Each question will have its own linear regression plot, a comparison table, an error summary, recovery actions used summary, and 2 legends on the linear regression plot. Legends for errors, the other for all queried data.
 - HTTP 432 Error = Request Signature Authentication Fails.
 - HTTP 404 Error = Not Found.
 - Recovery actions set in place:
 - The LLM-Agent tries to “fix” the input and try again.
 - Next, it has the capability to switch tools.
 - Lastly, it will just use the baseline LLM. This option is used last.

5.6.1 Finally, Some Code:

The benchmark calls three live APIs and rewrites the result files, so it only runs when explicitly asked for. By default this document is built from the committed results of the original run (36 queries), which keeps the analysis below reproducible without spending API quota.

To re-run the benchmark end to end – wiping the old .JSONL/.CSV files and regenerating them via ‘Agents_Multi_step.py’ – render with `RUN_BENCHMARK=1`.

Note that the original results were produced with Groq’s `llama-3.3-70b-versatile`, which has since been decommissioned. A re-run therefore uses a different model (set `GROQ_MODEL` in `.env`) and will not reproduce the exact numbers reported here.

```
import os

# The benchmark calls three live APIs (Groq, Tavily, OpenWeather) and
# overwrites the committed result files, so it is opt-in rather than
# automatic. Set RUN_BENCHMARK=1 to re-run it; otherwise this document
# is built from the committed results of the original run.
RUN_BENCHMARK = os.getenv("RUN_BENCHMARK", "").strip().lower() in {"1", "true", "yes"}

RESULT_FILES = [
    "agent_runs.jsonl",
    "baseline_runs.jsonl",
    "agentic_results.csv",
    "baseline_results.csv",
]

def reset_logs():
    """Delete old log files so a fresh benchmark starts clean."""
    for f in ["agent_runs.jsonl", "baseline_runs.jsonl",
```

```

        "agent_runs.csv", "baseline_runs.csv"]]:
    if os.path.exists(f):
        os.remove(f)
        print(f"Deleted: {f}")

if RUN_BENCHMARK:
    print(f"RUN_BENCHMARK set - running the full live benchmark.")
    print(f"Model: {os.getenv('GROQ_MODEL', 'openai/gpt-oss-120b')}")
    reset_logs()

    from Agents_Multi_step import run_full_benchmark_and_save_csv
    agentic_stats, baseline_stats = run_full_benchmark_and_save_csv()
else:
    missing = [f for f in RESULT_FILES if not os.path.exists(f)]
    if missing:
        raise FileNotFoundError(
            "Committed result files are missing: " + ", ".join(missing)
            + ". Re-generate them with: RUN_BENCHMARK=1 quarto render LLM-Agents.qmd"
        )
    print("Using committed benchmark results.")
    print("Set RUN_BENCHMARK=1 to re-run the live benchmark instead.")
    print()
    for f in RESULT_FILES:
        print(f"  {f}: {os.path.getsize(f):,} bytes")

```

Using committed benchmark results.

Set RUN_BENCHMARK=1 to re-run the live benchmark instead.

```

agent_runs.jsonl: 69,778 bytes
baseline_runs.jsonl: 12,501 bytes
agentic_results.csv: 17,227 bytes
baseline_results.csv: 5,721 bytes

```

5.7 'Does the complexity(# of tool calls required to complete a task) of the task predict total_latency?

```

from LLM_Agent_pipeline import run_regression_comparison
run_regression_comparison(
    predictor="task_complexity",
    outcome="total_latency"
);

```


=== Comparison Table ===

	Metric	LLM-Agent	Baseline
0	Slope (predictor)	0.2258	0.0000
1	Intercept	0.2456	0.2828
2	p-value (slope)	0.0476	NaN
3	R-squared	0.1207	-0.0000
4	Adj R-squared	0.0924	-0.0000
5	N	33.0000	36.0000

=== LLM-Agent Error Summary (grouped) ===

```

error_group
HTTP 432          51
Syntax error      23
HTTP 404          11
Invalid character   7
Invalid number     2
Name: count, dtype: int64

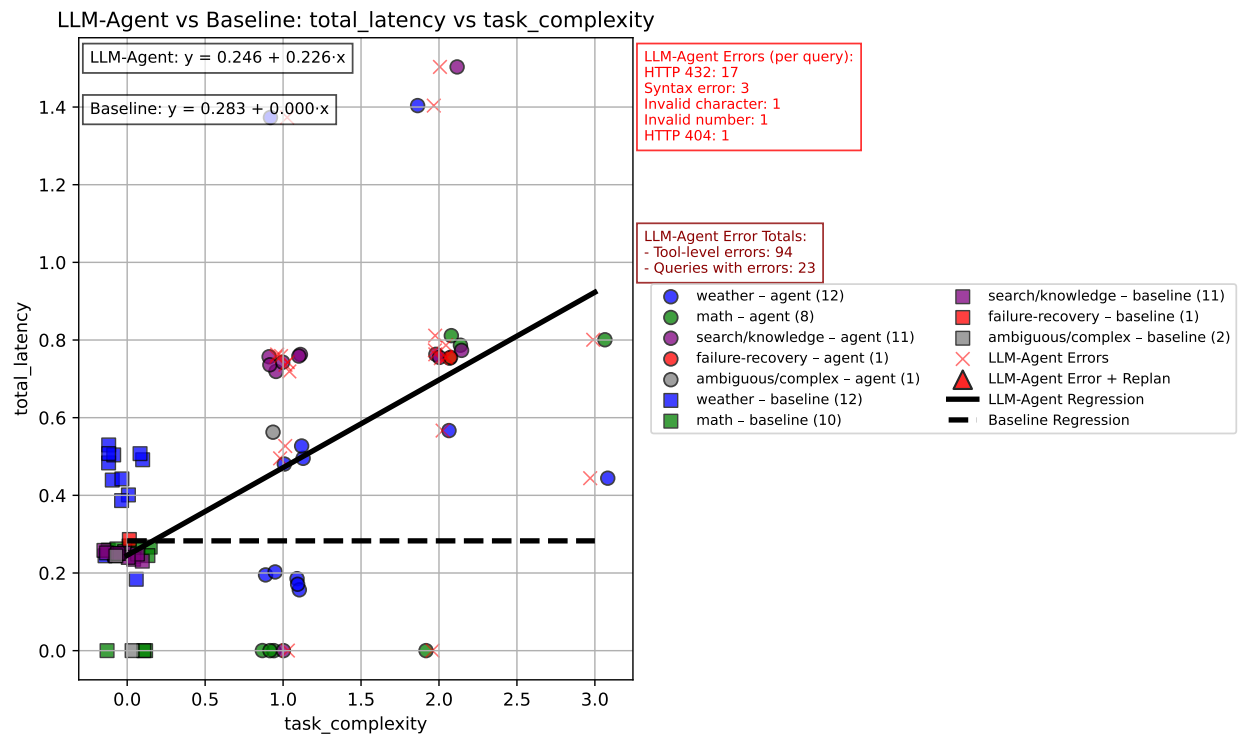
```

=== LLM-Agent Recovery Actions Used ===

```

recovery_action
replan          92
no_recovery       2
Name: count, dtype: int64

```



5.8 'Does the number of tools(actual # of tool calls the agent executed) used predict total task latency?

```
run_regression_comparison(  
    predictor="tool_count",  
    outcome="total_latency"  
);
```

=== Comparison Table ===

	Metric	LLM-Agent	Baseline
0	Slope (predictor)	0.1229	0.0000
1	Intercept	0.1196	0.2828
2	p-value (slope)	0.0001	NaN
3	R-squared	0.3905	-0.0000
4	Adj R-squared	0.3708	-0.0000
5	N	33.0000	36.0000

=== LLM-Agent Error Summary (grouped) ===

error_group

HTTP 432	51
Syntax error	23
HTTP 404	11
Invalid character	7
Invalid number	2

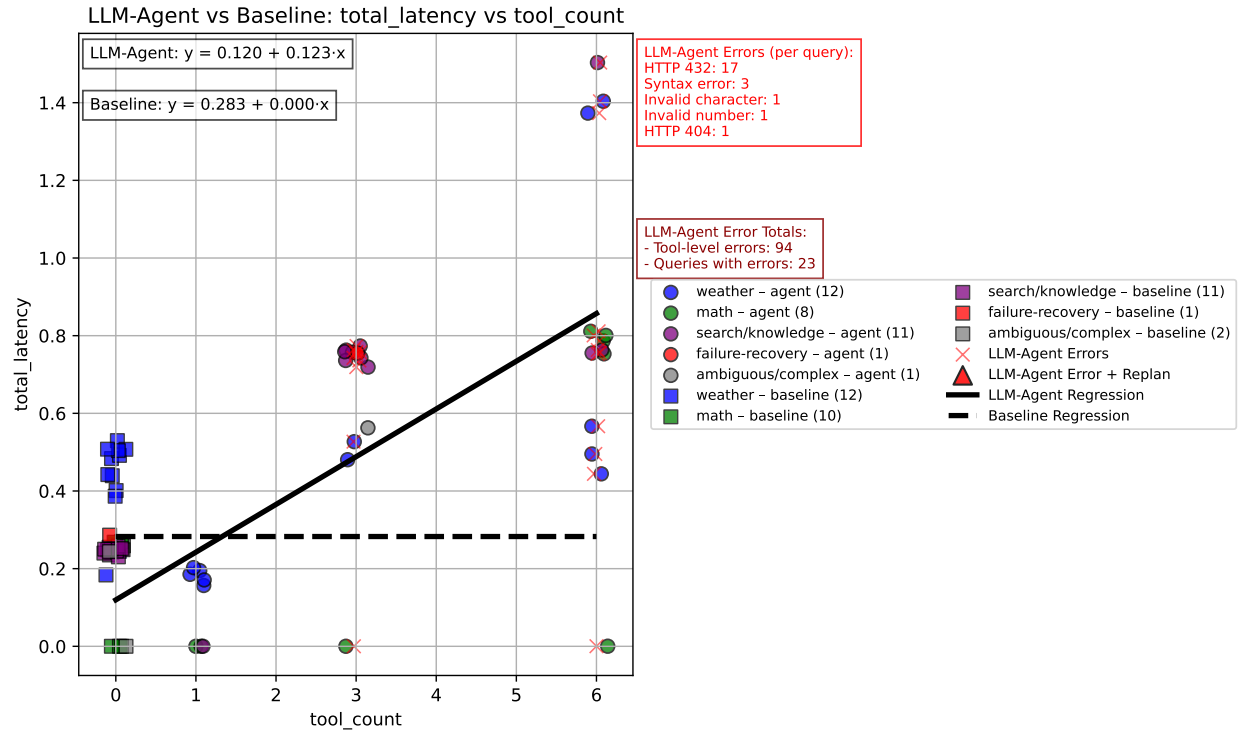
Name: count, dtype: int64

=== LLM-Agent Recovery Actions Used ===

recovery_action

replan	92
no_recovery	2

Name: count, dtype: int64



5.9 Does API difficulty(Calculator:1, Web search: 2, Weather: 3) predict total_latency?

```
run_regression_comparison(
    predictor="api_difficulty",
    outcome="total_latency"
);
```

=== Comparison Table ===

	Metric	LLM-Agent	Baseline
0	Slope (predictor)	0.2344	0.0000
1	Intercept	-0.2216	0.2828
2	p-value (slope)	0.0004	NaN
3	R-squared	0.3405	-0.0000
4	Adj R-squared	0.3192	-0.0000
5	N	33.0000	36.0000

=== LLM-Agent Error Summary (grouped) ===

error_group	
HTTP 432	51
Syntax error	23
HTTP 404	11
Invalid character	7

```
Invalid number          2
Name: count, dtype: int64
```

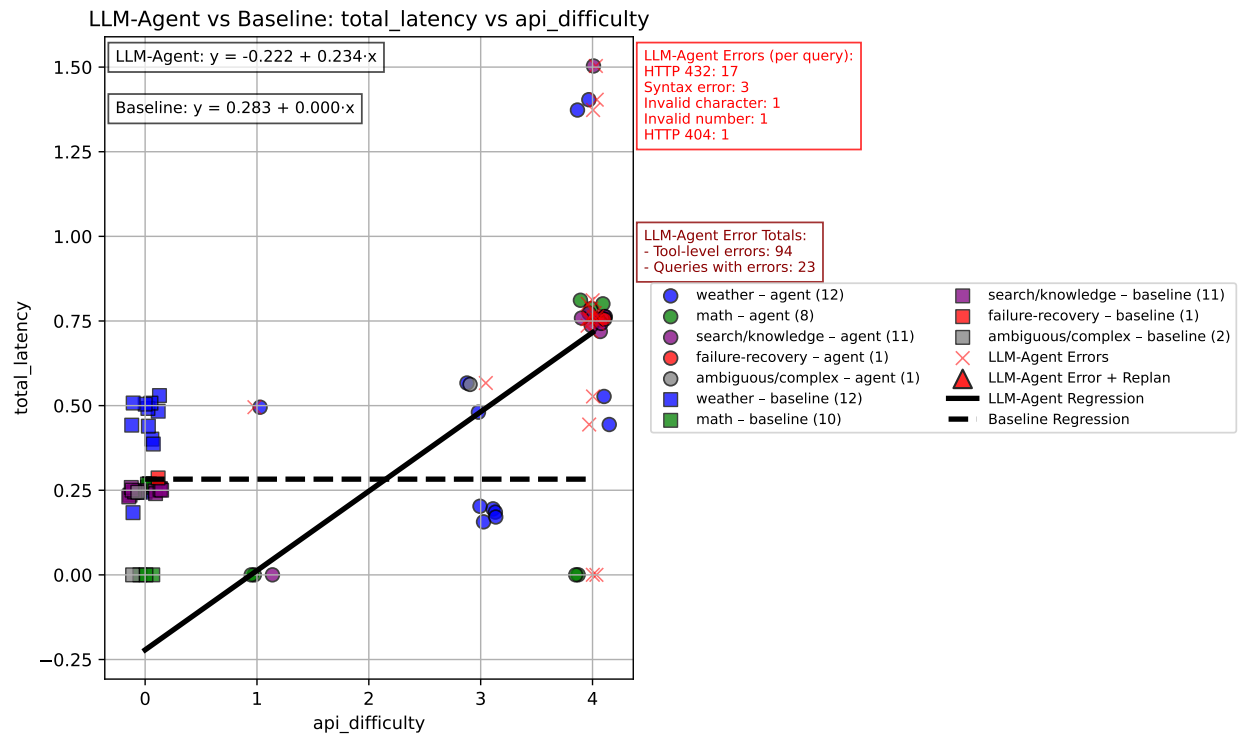
=== LLM-Agent Recovery Actions Used ===

```
recovery_action
```

```
replan          92
```

```
no_recovery      2
```

```
Name: count, dtype: int64
```



6 Conclusion:

The comparison between LLM-Agents and a baseline LLM is clear. This data shows how multi-step reasoning, tool use, recovery actions, and API use can change how effective a LLM can be. You can clearly see how the LLM-Agent is able to handle errors, self recover, access API keys and do multi-step reasoning and use all of that to predict total latency. I hope I showed just how effective an LLM-Agent can be. Which one would you choose?

7 For those wanting more!

- https://www.vellum.ai/llm-leaderboard?utm_source=bing&utm_medium=organic (compares top LLM models and LLM-Agents)
- <https://www.thinkstack.ai/blog/what-are-llm-agents/>

8 Disclaimer:

Remember to use any AI tool responsibly. AI is a wonderful tool but can be very easy to rely on it! Do not blindly trust the AI! Use those critical problem solving skills on hand!

9 Questions, comments, suggestions?

References:

- Ferrag, M. A., Tihanyi, N., & Debbah, M. (2025). From LLM reasoning to autonomous AI agents: A comprehensive review. *arXiv Preprint arXiv:2504.19678*. <https://arxiv.org/abs/2504.19678>
- Zhu, J., Zhu, M., Rui, R., Shan, R., Zheng, C., Chen, B., Xi, Y., Lin, J., Liu, W., Tang, R., Yu, Y., & Zhang, W. (2025). Evolutionary perspectives on the evaluation of LLM-based AI agents: A comprehensive survey. *arXiv*. <https://arxiv.org/abs/2506.11102>